

CMPEN-441: Operating System and System Programming

Homework 13: I/O Devices

Objectives

By completing this assignment, students should know:

- System architecture
- Communications between the OS and I/O devices

Implementation (20 points)

To test if your study meets the objectives, you are required to answer the following questions.

Question 1(2 points)

Basic Concept

What is the principal design rule for the modern system?

Answer: **Separate mechanism from policy**

Question 2(2 points)

Basic Concept

What is the polling in a canonical device?

Answer: **CPU checks a device's status register in a loop to determine if ready for I/O**

Question 3(2 points)

Case study

Compare the following two options to use the CPU; which one is more efficient?

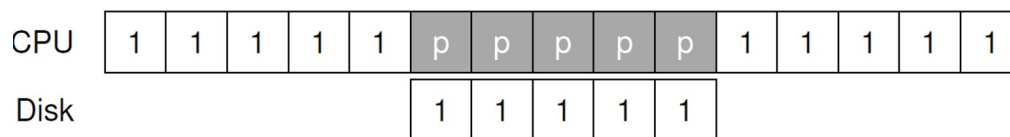
Why?

P: polling / spins

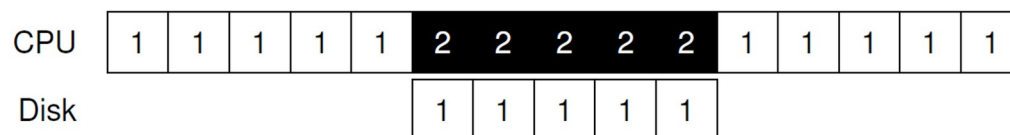
1: Process 1

2: Process 2

Option One:



Option Two:



Answer: 2. avoids waiting

Question 4(2 points)

Basic Concept

Do you think polling (spins) is efficient for using CPU resources? If not, do you have an alternative?

Answer: No. DMA?

Question 5 (2 points)

Basic Concept

How does the interruption work?

Answer: A device sends an interrupt signal to the CPU when it needs attention. CPU stops current exec, saves state, executes interrupt handler, returns to prev. task

Question 6(2 points)

Basic Concept

What is the DMA (Direct Memory Access)? What is the purpose of using the DMA?

Answer: DMA allows devices to transfer data directly to/from memory without CPU involvement. Reduces CPU overhead.

Question 7 (8 points)

Case Study

In the following code of the xv6 IDE disk driver, answer the following questions:

1) Is the data read from or written into the disk?

Answer: depends on b→flags &B_DIRTY

2) What is the size of data for the data traffic?

Answer: 512b

3) which method is used for data transmission, the polling or the interruption?

Answer: interrupt

4) To protect the data traffic, which mechanism(s) is(are) used: the mutex-lock, the condition variable, or the semaphores?

Answer: mutex lock + cond var

Submission

Follow the name convention shown below and submit the file(s) to Canvas before the deadline:

- **HW12_YourFirstnameLastname.doc**
OR HW12_YourFirstnameLastname.docx

```

static int ide_wait_ready() {
    while (((int r = inb(0x1f7)) & IDE_BSY) || !(r & IDE_DRDY))
        ; // loop until drive isn't busy
}

static void ide_start_request(struct buf *b) {
    ide_wait_ready();
    outb(0x3f6, 0); // generate interrupt
    outb(0x1f2, 1); // how many sectors?
    outb(0x1f3, b->sector & 0xff); // LBA goes here ...
    outb(0x1f4, (b->sector >> 8) & 0xff); // ... and here
    outb(0x1f5, (b->sector >> 16) & 0xff); // ... and here!
    outb(0x1f6, 0xe0 | ((b->dev&1)<<4) | ((b->sector>>24)&0x0f));
    if(b->flags & B_DIRTY){
        outb(0x1f7, IDE_CMD_WRITE); // this is a WRITE
        outsl(0x1f0, b->data, 512/4); // transfer data too!
    } else {
        outb(0x1f7, IDE_CMD_READ); // this is a READ (no data)
    }
}

void ide_rw(struct buf *b) {
    acquire(&ide_lock);
    for (struct buf **pp = &ide_queue; *pp; pp=(*pp)->qnext)
        ; // walk queue
    *pp = b; // add request to end
    if (ide_queue == b) // if q is empty
        ide_start_request(b); // send req to disk
    while ((b->flags & (B_VALID|B_DIRTY)) != B_VALID)
        sleep(b, &ide_lock); // wait for completion
    release(&ide_lock);
}

void ide_intr() {
    struct buf *b;
    acquire(&ide_lock);
    if (!(b->flags & B_DIRTY) && ide_wait_ready(1) >= 0)
        insl(0x1f0, b->data, 512/4); // if READ: get data
    b->flags |= B_VALID;
    b->flags &= ~B_DIRTY;
    wakeup(b); // wake waiting process
    if ((ide_queue = b->qnext) != 0) // start next request
        ide_start_request(ide_queue); // (if one exists)
    release(&ide_lock);
}

```

Figure 36.5: The xv6 IDE Disk Driver (Simplified)